# Home assignment - Research Analyst

## Part 1 - Memory Management

### The Scenario

You are part of a Red Team assessing the security of a legacy hypervisor used in a multi-tenant cloud environment. Our hypothesis is that the memory management system uses a **deterministic eviction algorithm**, which is a potential security vulnerability.

If a malicious tenant can predict exactly which physical memory frame will be freed next, they can corrupt data or even launch a Denial of Service (DoS) attack against other tenants sharing the same physical RAM.

Your task is to model this system, demonstrate the vulnerability, and propose a secure patch.

### Part 1.1: The Simulation (System Modeling)

**Goal:** Build a working model of the victim system to test your theories.

Implement a simulation in Python using a **Direct Mapped** memory model.

- **The Constraint:** Physical RAM is a fixed-size array of `NUM_FRAMES`.
- **The State:** You must track which **Virtual Page Number** is currently stored in each **Physical Frame Index**.
- **The Logic:**
    1. **Access:** When a virtual page is requested, check if it is already in RAM.
    2. **Hit/Miss:** If it is in RAM, it is a "Hit". If not, it is a "Miss" (Page Fault).
    3. **Eviction:** If RAM is full during a "Miss", the system must **evict** (remove) an existing page to make room for the new one.

**The Vulnerability (Eviction Logic):** When choosing which frame to evict, the system calculates the "Cyclical Distance" between the page being swapped *in* (`P_in`) and every currently loaded page (`P_resident`). It evicts the page that maximizes this distance:

```
None
Distance = min(abs(P_in - P_resident), NUM_PAGES - abs(P_in - P_resident))
```

*Note: If multiple pages have the same maximal distance, evict the one with the smallest Frame Index.*

## Part 1.2: The Exploit (Vulnerability Analysis)

**Goal:** Prove the vulnerability exists by writing a script to exploit it.

**Technical Note (Read Before Starting):**

- **Abstraction:** This is a logical simulation. You are **not** required to manipulate actual system memory or lower-level drivers. "Memory" should simply be represented by a standard data structure (like a Python List/Array) in your code.
- **Execution:** "Accessing Memory" is simply a function call in your script (e.g., `simulation.access(page_id)`). Do not over-engineer the environment; we are testing your understanding of the algorithm and the math.

**The Setup:**

- **The Target:** A high-value "Secret Key" is stored in **Virtual Page 50**. It is currently resident in physical RAM.
- **The Environment:** Physical RAM is 100% full.
- **The Attacker:** You control a malicious process that can request its own pages.

**The Task:** Write a function `generate_attack_sequence(target_page)` that determines the exact Virtual Page number your malicious process should request to **force the system to evict Page 50**.

**Required Output:** Produce a log from your simulation showing:

1. **RAM State:** `[..., Page 50, ...]` (Page 50 is present)
2. **Attacker Action:** `Requesting Page X`
3. **System Response:** `Evicting Page 50` (Target Hit)

## Part 1.3: The Fix (Mitigation Strategy)

**Goal:** Propose a solution that balances Security with Performance.

1. **The Patch:** Modify the eviction logic in your simulator to introduce **entropy** (unpredictability). The goal is to make it impossible for your exploit from Part 2 to reliably target Page 50.
2. **The Benchmark:**
    - **Security Check:** Run your "Exploit" sequence from Part 2 against your patched code. Prove that Page 50 is *no longer* reliably evicted.
    - **Performance Check:** Compare your **Patched Algorithm** vs. the **Original Algorithm** on a trace of 1,000 random memory accesses. Does your patch significantly increase the "Page Fault Rate"? (Ideally, the performance penalty should be minimal).

### Submission Guidelines

Please submit a `.zip` file containing:

1. **`simulation.py`**: Your simulation code (including the original and patched logic).
2. **`attack_demo.py`**: The script that runs the exploit demonstration.
3. **`Report.pdf`** (1-2 pages) containing:
    - **Vulnerability Summary:** Briefly explain *why* the original algorithm allowed you to reliably target Page 50.
    - **Mitigation Logic:** Explain your fix.
    - **Results:** A simple table comparing the Page Fault counts of the two algorithms.

# Part 2 - IAM Policy Classification Engine

## Objective

At Upwind, we explore AI-driven security automation. In this task, you will develop an **AI-based classification engine** that evaluates **IAM policies written in JSON format** and determines whether they are **weak or strong** using a Large Language Model (LLM).
This task will assess your ability to:

- Integrate AI models (Hugging Face, OpenAI, Amazon Bedrock, Grok, OLlama, etc) into security-related workflows.
- Process structured security policies and classify them based on risk.
- Improve AI-generated classifications through prompt refinement.

## Task Overview

Your task is to **build an AI-powered engine** that:

1. **Accepts JSON IAM policies as input**
2. **Uses an LLM to classify policies** as "Weak" or "Strong"
3. **Provides an explanation** for the classification
4. **Returns structured JSON output**

This engine will help automate IAM policy reviews by leveraging AI models for classification.

## Steps to Complete the Task

1. Build the AI Classification Engine
    1.1. Use an **LLM API** (e.g., OpenAI, Hugging Face, Amazon Bedrock).
    1.2. Write a script that:
        1.2.1. Takes a **JSON IAM policy** as input.
        1.2.2. Calls an LLM to **analyze and classify** the policy.
        1.2.3. Outputs **structured JSON results** with a classification and justification.
2. Test the Engine with Example Policies

**Example 1: Weak Policy**

**Description:** This policy allows **anyone** to perform **any action** on **any resource**, with no restrictions. This means a user could delete databases, modify security settings, or access sensitive data without any constraints.

**Input JSON Policy:**

```
None
{
  "Version": "2022-10-17",
```

```
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
}
```

**Expected AI Output:**

```
None
{
  "policy": {
    "Version": "2022-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": "*",
        "Resource": "*"
      }
    ]
  },
  "classification": "Weak",
  "reason": "This policy grants full unrestricted access to all resources and
actions, posing a significant security risk."
}
```

**Example 2: Strong Policy**

**Description:** This policy **only allows administrators** to **delete objects** in a specific **S3 bucket** and enforces **Multi-Factor Authentication (MFA)** before granting access. This adds **accountability** and **limits access** to only necessary actions.

**Input JSON Policy:**

```
None
{
  "Version": "2022-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:DeleteObject",
      "Resource": "arn:aws:s3:::secure-bucket/*",
      "Condition": {
        "Bool": { "aws:MultiFactorAuthPresent": "true" }
      }
    }
  ]
}
```

**Expected AI Output:**

```
None
{
  "policy": {
    "Version": "2022-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": "s3:DeleteObject",
        "Resource": "arn:aws:s3:::secure-bucket/*",
        "Condition": {
          "Bool": { "aws:MultiFactorAuthPresent": "true" }
        }
      }
    ]
  },
  "classification": "Strong",
  "reason": "The policy enforces security best practices by limiting actions,
scoping access to a specific resource, and requiring MFA."
}
```

## 3. Improve AI Classification Accuracy

- **Test multiple LLMs** and compare responses.
- **Refine prompts** to improve classification accuracy.
- **Identify patterns where AI misclassifies policies** and adjust accordingly.

### Deliverables

#### 1. AI Classification Engine (Python Script)

- A code (script) that:
  - Accepts **IAM policies in JSON format**.
  - Calls an LLM API for classification.
  - Outputs results in **structured JSON format**.

#### 2. Result Summary

- Screenshots of the execution
- Reasoning result and output

## Part 3 - Security Risk Assessment - Bonus

You are presented with three common cloud security issues. For each issue, provide a thorough risk assessment and remediation strategy.

### Security Issues to Analyze

1. **Unrestricted Network Access via Security Groups or Firewall Rules**
   **Example:** A virtual machine or managed service is protected by a security group or firewall rule that allows inbound traffic from 0.0.0.0/0 on sensitive ports such as SSH (22), RDP (3389), or database ports.

2. **Unauthenticated Application Exposure**
   **Example:** An internal or customer-facing application endpoint is deployed without authentication or authorization controls, allowing anonymous users to access sensitive functionality, internal APIs, or administrative interfaces.

3. **Source Code Artifacts Containing Embedded Git Tokens**
   **Example:** Build artifacts, container images, or packaged application binaries that include embedded Git access tokens or credentials, enabling unauthorized access to private repositories or CI/CD pipelines if extracted.

### Deliverables

For each of the three issues, you must address:

1. Main Security Issue
   a. Describe the nature of the vulnerability or misconfiguration.
   b. Provide relevant details on why this is considered a critical or high-risk problem.
2. Attacker Motivation and Potential Risk

    a. Explain how attackers could exploit the issue.
    b. Assess the impact or severity if the issue is abused (data theft, privilege escalation, account compromise, financial loss, etc.).
3. Recommendations for Remediation
    a. List specific actions the customer should take to fix or mitigate the issue.
4. Verification and Ongoing Validation
    a. Detail how you would test or audit the environment to ensure the vulnerability has been resolved.
    b. Include tips or tools (security scanning software, cloud provider logs, configuration management frameworks) that can be used to detect or prevent a recurrence of the same issue.